AIA2: Advanced Mathematics for AI

Module Overview and Theoretical Foundations

Advanced mathematics provides the rigorous foundation for training AI systems that generalize beyond training data. This module bridges **optimization theory** (escaping non-convex traps) with **information theory** (quantifying learning progress), enabling practitioners to diagnose failures and design superior algorithms.

Core Learning Objectives:

- Derive convergence guarantees for optimizers
- Implement state-of-the-art optimizers from scratch
- Understand information-theoretic loss design
- Apply mathematical diagnostics to real training runs

Theoretical Framework: AI optimization lives in **non-convex stochastic bilevel problems**:

text

$\min_\theta E[\ell(f_\theta(x), y)]$ s.t. f_θ solves inner objectives

Prerequisites: Multivariable calculus, basic probability, matrix calculus

(Figure 1: Loss landscape pathologies - saddles, ravines, plateaus)

Section 1: Optimization Landscape Theory

Critical Points Classification:

text

$\nabla L = 0$ defines stationary points

Hessian $H = \nabla^2 L$ determines type:

• All $\lambda_i > 0$: local minimum

• Some $\lambda_i < 0$: saddle

• $\lambda_{max}(H) \approx 0$: flat minimum

High-Dimensional Geometry (Soudry 2018):

text

Dim ≥ 50: 99% critical points are saddles

Sharp minima generalize worse (Keskar 2016)

Escape Time Analysis:

text

Saddle escape requires noise $\propto \sqrt{(\text{curvature gap})}$

Diagnostic Tools:

text

1. Hessian-vector products: $\nabla^2 L\, v \approx \nabla(\nabla L \cdot v)$

2. Scale sharpness: max eigenvalue via Lanczos

3. Loss curvature: trace(H)/dim

Section 2: Gradient Descent - Complete Theory and Implementation

Vanilla Gradient Descent:

text

$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$

Convergence Theorems:

1. **Lipschitz smooth (∇L continuous):**

text

$||\nabla L(\theta)|| \leq L\, ||\theta - \theta^*||$

1. **Strongly convex ($\mu > 0$):**

text

$L(\theta) \geq L(\theta^*) + \frac{1}{2}\mu||\theta - \theta^*||^2$

text

Linear convergence: $||\theta_t - \theta^*|| \leq (1 - \mu/L)^t\, ||\theta_0 - \theta^*||$

Optimal $\eta = 2/(\mu + L)$

Stochastic GD Analysis (Bottou 2018):

text

$E[L(\theta_T)] - L^* \leq O(\sigma^2/(b\, T) + L\, ||\theta_0 - \theta^*||^2 / T)$

σ^2 = gradient noise, b = batch size

Momentum Methods:

Heavy-ball (Polyak 1964):

text

$v_{t+1} = \beta v_t + \nabla L(\theta_t)$

$\theta_{t+1} = \theta_t - \eta v_{t+1}$

Accelerates by 2-3x on quadratic bowls.

Nesterov Accelerated Gradient:

text

y_{t+1} = θ_t - β (θ_t - θ_{t-1})

v_{t+1} = β v_t + ∇L(y_{t+1})

θ_{t+1} = θ_t - η v_{t+1}

Production SGD Implementation:

python

```python
import torch

class SGDMomentum:

    def __init__(self, params, lr=0.01, momentum=0.9,

                 weight_decay=1e-4, dampening=0):

        self.defaults = {'lr': lr, 'momentum': momentum,

                         'weight_decay': weight_decay, 'dampening': dampening}

        self.state = {}


    def step(self, closure=None):

        loss = None

        if closure is not None:

            loss = closure()


        for group in self.param_groups:

            momentum = group['momentum']

            dampening = group['dampening']

            wd = group['weight_decay']


            for p in group['params']:

                if p.grad is None: continue


                state = self.state.setdefault(p, {})

                if 'momentum_buffer' not in state:

                    state['momentum_buffer'] = torch.zeros_like(p)

                    state['step'] = 0
```

```
            grad = p.grad.data
            buf = state['momentum_buffer']

            # Weight decay
            if wd != 0:
                grad = grad.add(p.data, alpha=wd)

            # Momentum
            buf.mul_(momentum).add_(grad, alpha=1-dampening)
            p.data.add_(buf, alpha=-group['lr'])

            state['step'] += 1
        return loss
```

Section 3: Adaptive Optimizers - Adam Family Deep Dive

RMSProp (precursor):

text

$E[g^2]_t = \rho E[g^2]_{t-1} + (1-\rho) g_t^2$

$\theta_t = \theta_{t-1} - \eta g_t / (\sqrt{E[g^2]_t + \varepsilon})$

Adam Complete Derivation:

text

$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$

$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$

Bias correction (finite sum geometric series):

$\hat{m}_t = m_t / (1 - \beta_1^t)$

$\hat{v}_t = v_t / (1 - \beta_2^t)$

$\theta_t = \theta_{t-1} - \alpha \hat{m}_t / (\sqrt{\hat{v}_t + \varepsilon})$

Theoretical Guarantees (Reddi 2018):

text

Adam converges on convex problems

May diverge without proper $\beta_2 < 1$

AdamW - The Production Standard:

text

Decoupled weight decay (Loshchilov 2017)

θ_t = θ_{t-1} - α m̂_t / √v̂_t # Adam step

θ_t = θ_t * (1 - α λ) # Decay step

Hyperparameter Analysis:

text

$\beta_1 \in [0.9, 0.95]$: momentum

$\beta_2 \in [0.98, 0.9999]$: variance timescale

ε = 1e-8: numerical stability

Complete AdamW Implementation:

python

```python
class AdamW:
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8,
                 weight_decay=0.01, amsgrad=False):
        defaults = dict(lr=lr, betas=betas, eps=eps, weight_decay=weight_decay, amsgrad=amsgrad)
        super().__init__(params, defaults)

    def step(self, closure=None):
        loss = None
        if closure is not None:
            loss = closure()

        for group in self.param_groups:
            for p in group['params']:
                if p.grad is None: continue
                grad = p.grad.data
                state = self.state.setdefault(p, {})
```

```
# State init
if 'step' not in state:
    state['step'] = 0
    state['exp_avg'] = torch.zeros_like(p.data)
    state['exp_avg_sq'] = torch.zeros_like(p.data)
    if group['amsgrad']:
        state['max_exp_avg_sq'] = torch.zeros_like(p.data)

exp_avg, exp_avg_sq = state['exp_avg'], state['exp_avg_sq']
beta1, beta2 = group['betas']

state['step'] += 1

# Decay step first
p.data.mul_(1 - group['lr'] * group['weight_decay'])

# Adam steps
exp_avg.mul_(beta1).add_(grad, alpha=1 - beta1)
exp_avg_sq.mul_(beta2).addcmul_(grad, grad, value=1 - beta2)

if group['amsgrad']:
    max_exp_avg_sq = state['max_exp_avg_sq']
    torch.maximum(max_exp_avg_sq, exp_avg_sq, out=max_exp_avg_sq)
    denom = max_exp_avg_sq.sqrt()
else:
    denom = exp_avg_sq.sqrt()

bias_correction1 = 1 - beta1 ** state['step']
bias_correction2 = 1 - beta2 ** state['step']
```

```
            step_size = group['lr'] * math.sqrt(bias_correction2) / bias_correction1

            p.data.addcdiv_(exp_avg, denom, value=-step_size)

    return loss
```

Section 4: Cutting-Edge Optimizers and Scaling

Lion Optimizer (Google 2023):

text

$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$

$u_t = \beta_2 u_{t-1} - \eta \, \mathrm{sign}(m_t)$

2x faster, 1/3 memory vs AdamW for LLMs.

Sophia (Gao 2024):

text

Hessian clipping: $\sqrt{(\mathrm{clip}(H gg^T, \tau))}$

Learning Rate Schedules - Mathematical Design:

Cosine Annealing with Warmup:

text

$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t / T))$

Warmup phase: $\eta_t = \eta_{max} * t / warmup_steps$

Square-Root Schedule (LLM scaling):

text

$\eta_t \propto 1 / \sqrt{(\min(t, T))}$

Implementation:

python

```
import math

class CosineScheduler:

    def __init__(self, optimizer, T_total, warmup_steps=0, lr_max=1e-3, lr_min=1e-6):

        self.optimizer = optimizer

        self.T_total = T_total

        self.warmup_steps = warmup_steps

        self.lr_max = lr_max
```

```
        self.lr_min = lr_min
        self.step_count = 0

    def step(self):
        self.step_count += 1
        t = self.step_count

        if t <= self.warmup_steps:
            lr = self.lr_max * t / self.warmup_steps
        else:
            progress = (t - self.warmup_steps) / (self.T_total - self.warmup_steps)
            lr = self.lr_min + 0.5 * (self.lr_max - self.lr_min) * (1 + math.cos(math.pi * progress))

        for param_group in self.optimizer.param_groups:
            param_group['lr'] = lr
```

Scaling Laws Integration:

text

Optimal LR ∝ N^{-0.2} (for param count N)

Batch size scaling: linear up to 32k